CSC111 Project 2 Report: Minks

Caellum Yip Hoi-Lee, Catherine Abdul-Samad, Michael Chen, Joshua Yeung

March 26, 2026

1. Introduction and Research Question

Personal knowledge management (PKM) tools such as Obsidian([**?**]) implement the Zettelkasten methodology([**?**]), allowing users to organize their knowledge as a network of interlinked Markdown notes. A note in the network can link to other notes in the network through internal links called "WikiLinks", forming a graph where the notes are vertices and the links are edges. The benefit of such a system lies in its ability to provide users with insights into the relationship between ideas, and build a stronger understanding of their notes. However, many users (including ourselves), tend to under-link their notes, leading to a lackluster knowledge network. For example, one of the ways one of our group members uses Obsidian is through a "hub-and-spoke" structure, where minimal note vertices known as "hubs" form a directory with WikiLinks to other notes in the same topic (e.g. a hub named "CSC111.md" with WikiLinks to ("Tree.md", "Recursion.md", etc.)). This however, defeats the purpose of Obsidian, failing to capture the lateral connections notes can have in the network which actually make a knowledge graph useful. For instance, a link from "Trees.md" to "Abstract Syntax Trees (CS).md" to "Abstract Syntax Trees (Linguistics).md".

Whilst manually reviewing every note to add missing links is a valuable learning activity, it becomes highly impractical as a PKM system grows to hundreds or thousands of notes.

"Can we use graph-based link prediction algorithms augmented with NLP-based semantic similarity to suggest meaningful missing connections in an Obsidian knowledge graph, and how do the algorithms' judgements compare with a user's own judgement of which notes should be linked?"

2. Datasets

To evaluate our prediction pipeline and avoid circular bias during parameter tuning, we used two distinct datasets:

- **Vault A (Tuning Set):** A personal Obsidian vault belonging to one of the group members, consisting primarily of undergraduate course notes. This dataset was exclusively used in the `MinkPredictor.fit()` method to grid-search the optimal weighting between structural and semantic similarity scores.
- **Vault B (Test Set):** The open-source digital garden of Jacky Zhao (`jzhao.xyz`). This vault contains over 700 notes spanning computer science, philosophy, and other topics. It provides a denser and more diverse graph, allowing us to test our algorithm's out-of-domain generalizability. We utilized the raw `.md` plaintext and internal WikiLinks from this set.

3. Computational Overview

Data Representation

We model an Obsidian vault using a custom `KnowledgeGraph` class, representing the vault as an undirected graph $G = (V, E)$. Each Markdown note is parsed by `load_graph.py` to strip YAML frontmatter and extract plaintext, becoming a `_Note` vertex $v \in V$. WikiLinks between notes become edges $\{u, v\} \in E$.

Major Computations

Our prediction pipeline, orchestrated by the `MinkPredictor` class, performs the following major computations:

1. **Structural Similarity:** We manually implemented the Jaccard index and the Adamic-Adar index. Adamic-Adar assigns higher weight to shared neighbours with fewer connections, capturing the intuition that a rare mutual connection is stronger evidence for a missing link than a highly connected "hub" note.

2. **Semantic Similarity:** To detect lateral connections between **structurally** isolated notes, we generate text embeddings. We built a manual **TF-IDF** (Term Frequency-Inverse Document Frequency) vectorizer from scratch to serve as a baseline, alongside an implementation using the `sentence-transformers` library for dense vector embeddings, which we use to calculate the Cosine Similarity between these vectors.

3. **Weight Tuning & Holdout Evaluation:** We compute a combined score $S = (w_{struct} \times \text{Structural}) + (w_{sem} \times \text{Semantic})$. We evaluate this using an edge holdout method: randomly removing 20% of existing edges, predicting the top-k missing links, and calculating `recall@k` and `precision@k`.

Visualization and New Libraries

We used three libraries to report our results interactively:

- **NetworkX:** We convert our custom graph object into a `networkx.Graph` in `visualize.py` to use its `spring_layout()` function, calculates 2D node coordinates based on structural edges.

- **Plotly:** We use `plotly.graph_objects` to render the calculated coordinates into an interactive HTML visualization, visually distinguishing minks' predicted links (red, dashed) from existing WikiLinks (grey, solid).

- **Scikit-Learn (t-SNE):** To compare structural versus semantic clustering, we use `sklearn.manifold.TSNE` to reduce our high-dimensional text embeddings into a 2D space, plotted side-by-side with the NetworkX layout.

4. Instructions for Running the Program

1. **Obtain Datasets:** Download the `vaults.zip` file from

   todoinsertlink

   . Extract the folder so that the `vaults/` directory (containing `vault_a` and `vault_b`) is in the same directory as the `src/` folder.

2. **Install Dependencies:** Run:
   `pip install -r src/requirements.txt`. This will install `plotly`, `networkx`, `scikit-learn`, and `sentence-transformer`

3. **Execute:** Run the main pipeline via:
   `python src/main.py`.

4. **Expected Output:** The console will print the progress of the weight-fitting grid search on Vault A, followed by the holdout evaluation metrics (Precision/Recall at k) for Vault B. It will output the top predicted missing links and graph statistics (centrality/components). Finally, it will generate two interactive HTML files in the `output/` directory: `vault_b_graph.html` and `layout_comparison.html`. Open these in a web browser to explore the visualizations.

5. Changes to Project Plan

Following feedback on our Phase 1 proposal, we made several critical adjustments to our methodology to improve computational complexity:

- **Preventing Circular Evaluation:**
- **Manual TF-IDF Implementation:** To ensure our project met the computational complexity requirements, we did not solely rely on the `sentence-transformers` library. We built a TF-IDF embedder from scratch within `similarity.py`, managing term frequencies, inverse document frequencies, and vector normalization natively.
- **Precision Metrics:** We added `precision@k` calculations alongside `recall@k`, ...
- **Semantic vs. Structural Visual Comparison:** We expanded our visualization pipeline to include t-SNE, directly comparing a layout generated from pure semantic embeddings against the force-directed structural layout.

6. Discussion

The results of our computational exploration provide a nuanced answer to our research question: graph-based algorithms augmented with semantic NLP can successfully identify missing connections, but the definition of a "meaningful" link remains highly subjective.

...